

Project Report - Task 2

Data Storage Paradigms, IV1351

2025/11

Group 27 Project members:

[Sam Serbouti, serbouti@kth.se]

[Evelina Fridmane, fridmane@kth.se]

Contents

1	Introduction	2
2	Literature Study	2
3	Method	2
4	Results	3
4.1	Query 1	3
4.2	Query 2	4
4.3	Query 3	5
4.4	Query 4	6
5	Discussion	7
5.1	Analysis of the planned hours query	7
5.2	Analysis of actual allocated hours for a course query	9
5.3	Analysis of the total allocated hours query	11
5.4	Analysis of the teacher course load query	12
6	Higher grade discussion	15
6.1	Indices for our workload queries	15
6.2	Materialized views	16
6.3	Improving our queries and analyzing improvements	17
6.3.1	For the first query	18
6.3.2	For the second query	18

6.3.3	For the third query	19
6.3.4	For the fourth query	19
6.3.5	For the fifth query	20
6.4	Self assessment	22
7	Comments About the Course	23

1 Introduction

This report presents the implementation of an OLAP - Online Analytical Processing, queries and views as well as an efficiency analysis of the efficiency of one of the queries with EXPLAIN ANALYZE. This is based on the relational database for university courses and teachers that was built in the previous task using PostgreSQL. In addition to these mandatory requirements, the specific workload of the provided queries is analyzed to determine the optimal use of indices, materialized views, etc.

2 Literature Study

Before completing the given task, we started with lectures videos on relational models and live lecture on SQL. Then to improve our understanding we read chapters 6 and 7 of the 7th edition of *Fundamentals of Database Systems* by Ramez Elmasri and Shamkant Navathe that were about SQL and RDBMS standards. These chapters provided us with fundamental knowledge for this task.

Online, we used the following resources:

- The official Postgresql documentation on CURRENT DATE, EXTRACT, and JOINS
- A GeeksForGeeks page on GROUP BY
- A page on EXPLAIN (ANALYZE) by Cybertec
- An article on shared buffers by fastware.com
- An article on EXPLAIN ANALYZE by pganalyze.com

3 Method

We are using PostgreSQL as our DBMS. For development tools we used pgAdmin4 and DBeaver, which allowed us to write, execute and visualize SQL queries in structured tables.

We have first had to modify our database population, since while testing our queries, we noticed that our different softwares created the IDs (INT GENERATED ALWAYS AS IDENTITY differently. To make sure that both us, the database administrators and any end-user would have the same reference, we modified our database creation script so that

we forced PostgreSQL¹ to have every ID start at 1. To verify that the SQL queries work as intended we first went through each part very carefully, especially checking JOINS as we wanted to avoid using nested queries. When we got the output we manually compared it with our database to ensure that everything is calculated correctly.

4 Results

The git repository linking to every SQL queries as well as our model can be found here. Our github repo shows all initial queries, their results, their output of `EXPLAIN ANALYZE`, their drawn execution plan as well as the improved or new queries (for the higher grade section) and their output of `EXPLAIN ANALYZE`.

4.1 Query 1

This first query sums, per course instance in the current year (of the year this query is being run), how many weighted teaching hours are planned for each activity type (Lecture, Tutorial, Lab, Seminar, Overhead). They are weighted by the multiplication factor given by the customer and that depend of the activity name which is why we use conditional sums, for example:

```
SUM(CASE WHEN ta.activity_name = 'Seminar'
        THEN pa.planned_nb_hours * ta.factor ELSE 0 END) AS "Seminar Hours",
SUM(CASE WHEN ta.activity_name = 'Overhead'
        THEN pa.planned_nb_hours * ta.factor ELSE 0 END) AS "Other Overhead Hours",
```

It also computes administration and examination hours using formulas that depend on course credits (hp) and number of students:

```
(2*cl.hp+28+0.2*ci.num_students) AS "Administration Hours",
(32+0.725*ci.num_students) AS "Examination Hours",
```

Finally, it sums all planned weighted hours plus admin and exam to give the total planned hours per course instance:

```
SUM(pa.planned_nb_hours*ta.factor)+(2*cl.hp+28+0.2*ci.num_students)+...
...(32+0.725*ci.num_students) AS "Total Hours"
```

For this query, we needed to perform inner-equijoins on multiple tables based on their IDs

The result of this query are:

¹using the command `INT GENERATED BY DEFAULT AS IDENTITY (START WITH 1 INCREMENT BY 1) PRIMARY KEY,`

Table 1: Planned Hours per Course Instance (Current Year) – Part 1

Code	Inst.	HP	Period	# Stud.	Lect.	Tut.	Lab	Sem.
100	1	7	P2	25	161.99	102.60	24.00	91.80
103	5	10	P2	25	122.40	61.20	33.60	61.20
107	9	10	P2	20	64.80	68.40	43.20	68.40
113	17	8	P2	15	93.60	46.80	62.40	46.80
119	13	10	P1	25	79.20	75.60	52.80	39.60

Table 2: Planned Hours per Course Instance (Current Year) – Part 2

Code	Inst.	Overh.	Admin	Exam	Total
100	1	15	47.00	50.13	492.52
103	5	1	53.00	50.13	382.52
107	9	1	52.00	46.50	344.30
113	17	6	47.00	42.88	345.47
119	13	4	53.00	50.13	354.32

4.2 Query 2

The main task of this query is to calculate the actual allocated hours for a course per teacher. Our implementation contains two queries: one small helper query and one main one. We made a decision to allocate Admin and Exam hours equally among the teachers/assistants in the course, as we weren't given any strict formula for distribution.

The first query is used to count the number of teachers allocated to a specific course to then allocate an equal amount of Admin and Exam hours to each teacher. In the main query, we select all the needed attributes from the database as well as calculated teaching activity hours using the provided factors. To calculate derived attributes such as Admin and Exam, we used the formula provided to us and divided by the number of teachers calculated previously. We joined all the required entities and then filtered the data for the current year and for course instance 1, which is Linear Algebra in P2.

Table 3: Teacher Allocation (Part 1: General Info & Direct Teaching)

Code	Inst.	HP	Teacher's Name	Designation	Lect.	Tut.	Lab	Sem.
100	1	7	Anna Andersson	Lab Assistant	54.00	0.00	24.00	0.00
100	1	7	Erik Berg	Dean	0.00	0.00	0.00	37.80
100	1	7	Filip Eriksson	Administrator	0.00	0.00	0.00	0.00
100	1	7	Jonas Johansson	Lab Assistant	0.00	54.00	0.00	0.00
100	1	7	Karin Ekström	Administrator	0.00	48.60	0.00	0.00
100	1	7	Maria Lindgren	Administrator	0.00	0.00	0.00	54.00
100	1	7	Niklas Olsson	Adjunct	108.00	0.00	0.00	0.00
100	1	7	Sven Lund	Technical Staff	0.00	0.00	0.00	0.00

Table 4: Teacher Allocation (Part 2: Overhead, Admin & Totals)

Teacher's Name	Overhead	Admin	Exam	Total Hours
Anna Andersson	2.00	5.88	6.27	92.14
Erik Berg	0.00	5.88	6.27	49.94
Filip Eriksson	1.00	5.88	6.27	13.14
Jonas Johansson	5.00	5.88	6.27	71.14
Karin Ekström	0.00	5.88	6.27	60.74
Maria Lindgren	0.00	5.88	6.27	66.14
Niklas Olsson	0.00	5.88	6.27	120.14
Sven Lund	7.00	5.88	6.27	19.14

4.3 Query 3

This query computes, per course instance, how many weighted teaching hours a specific teacher (in our case, we took Filip Eriksson) has in the current calendar year, broken down by activity type, and totals them. We first start by limiting the whole query by selecting the rows corresponding to the teacher whose name we set and restricting to the current year²:

$$\sigma_{P.first_name='Filip' \wedge P.last_name='Eriksson' \wedge CI.study_year=year(CURRENT_DATE)}$$

²Of when the query is being run.

Then we need to join³ the tables necessary to select the columns that interest us: person, employee, employee_activity, planned_activity, teaching_activity, course_instance and course_layout using equijoins on the IDs. Finally, we use conditional sums similarly to query 1.

The results of the query are as follows:

Table 5: Teacher Allocation for Filip Eriksson (Current Year)

Code	Inst.	HP	Teacher's Name	Lect.	Tut.	Lab	Sem.
100	1	7	Filip Eriksson	0.00	0.00	0.00	0.00
113	17	8	Filip Eriksson	93.60	0.00	0.00	0.00

4.4 Query 4

For this part of the task, we had to write a query that lists employee IDs and names of all teachers who are allocated to more than a specific number (we chose more than 0) of course instances during the current period. Here, we avoided using multiple queries and decided to calculate everything in one main query. We selected all needed attributes and counted the number of courses a teacher is allocated to using COUNT DISTINCT. After joining everything needed, we filtered the data by the current year and study period. We decided to take the period dates exactly as they are at KTH from the KTH website. Finally, we decided to use 0 as the specific number, so the provided data contains teachers having more than 0 courses at the current study period.

³Joins prevent us from using nested queries.

Table 6: Teachers Allocated to More Than 0 Courses (Current Period P2)

Employment ID	Teacher's Name	Period	No of courses
1	Anna Andersson	P2	1
2	Erik Berg	P2	2
3	Karin Ekström	P2	1
4	Oskar Nilsson	P2	1
5	Lisa Karlsson	P2	1
6	Jonas Johansson	P2	1
7	Sara Persson	P2	1
9	Eva Larsson	P2	1
10	Johan Lindberg	P2	1
11	Maria Lindgren	P2	1
12	Filip Eriksson	P2	2
14	Fredrik Danielsson	P2	1
15	Emma Holm	P2	1
16	Niklas Olsson	P2	1
17	Isabel Magnusson	P2	1
18	Sven Lund	P2	1
19	Nina Nordin	P2	1
20	Axel Westin	P2	1

5 Discussion

5.1 Analysis of the planned hours query

The statistics per node type

The operations executed for this query are:

- aggregate (GroupAggregate) = 1 (performs all grouping and sum calculations for the output rows);
- sort = 1 (sorts the results by course code, instance, and hp as required by GROUP BY and ORDER BY)
- nested loop = 4 (correspond to the joins of the four tables: *planned_activity* → *course_instance* → *course_layout* → *teaching_activity* where each join introduces another nested loop join node)
- seq scan = 1 (on *planned_activity*)
- index scan = 3 (for *course_instance*, *course_layout*, and *teaching_activity* for efficient row retrieval)

- `memoize = 1` (to cache instance lookups for repeated values of `pa.instance_id`)

For most joins, PostgreSQL used an index on the joined column (index scan), so it can jump directly to matching rows instead of reading whole tables. The only table it read fully, row by row, was `planned_activity` (sequential scan), which is fine because it's small. It then used quicksort to order the 31 intermediate rows before the `GROUP BY/ORDER BY`, which would become smart for a larger database.

The Memoize⁴ node caches `course_instance` lookups for each `instance_id`, so when the same `instance_id` appears again it reuses the cached result instead of doing another index scan, avoiding redundant work.

The statistics per relation

The accessed tables are: `planned_activity` (scanned sequentially once), `course_instance` (efficiently looked up by index scan, filtered on year), `course_layout` (index scan by foreign key), and `teaching_activity` (index scan by `teaching_activity_id`).

This matches expectations since each planned activity must be matched to the correct course instance and layout. The sequential scan on `planned_activity` is efficient for a small table; if it grew, an index by `instance_id` might help. The memoization hit/miss (80 hits, 20 misses) show that the same `instance_ids` are being reused many times, so it is saving work.

Performance insight step by step

Please see our execution plan tree in our github.

The planner starts from the `planned_activity` table and joins, by index, into the other tables. Filtering by year is done on `course_instance`. Sorting and aggregation are performed at the end, with group totals and column breakdowns produced. Overall, the path is efficient due to the many uses of indexes (on `teaching_activity`, `course_layout` and `course_instance`) and memory, with execution and planning times well under 2 ms for this dataset.

Adding an index

Right now the heavy work is:

- Seq scan on `planned_activity pa`
- index lookup on `course_instance ci` using `instance_id` (already indexed) lookups on `course_layout` and `teaching_activity` via their PKs (already exist)

The filter is on `course_instance.study_year`, but it's applied inside the index scan on `course_instance` (not on `planned_activity`), so a multi-column index on `course_instance(instance_id,`

⁴Memoize is a plan node in PostgreSQL that acts like a small cache in a nested loop join: it stores the results for each parameter value the first time it's looked up; if the same value appears again, it returns the cached rows instead of running the index scan again. This reduces repeated work.

study_year) would likely be redundant: instance_id is unique, and the condition on study_year barely filters after that.

If anything grows large, the candidate place for an index is planned_activity.instance_id, since that's the join column we currently seq-scan on. A simple index there is the first thing we could consider:

```
CREATE INDEX idx_planned_activity_instance ON planned_activity (instance_id);
```

A multi-column index on (instance_id, teaching_activity_id) would only help if we frequently filter on both columns together in WHERE or JOIN but this query doesn't, so a single-column index on instance_id is enough and cheaper to maintain.

Exploring views

A view on this query would mainly be for convenience or for reuse (easier to just write SELECT * FROM planned_hours.yyyy), not for speed by itself. It's useful if many reports reuse exactly this logic and we want one central place to change formulas.

5.2 Analysis of actual allocated hours for a course query

The statistics per node type

The operations executed for this query are:

- aggregate (GroupAggregate) = 2 (one for the main grouping of the final result, and one nested inside for the teacher count calculation);
- sort = 2 (used to order data with a quicksort before the aggregation steps);
- nested loop = 5 (the primary join method used to link the summarized data to employee, person, and job_title);
- hash join = 3 (used to efficiently join the larger activity tables: planned_activity, employee_activity, and teaching_activity);
- seq scan = 5 (on employee_activity twice, planned_activity twice, and teaching_activity once);
- index scan = 5 (on course_instance, course_layout, employee, person, and job_title).

The execution plan for this query is more complex because it combines two logical parts: a table to count the teachers and the main query to calculate hours. PostgreSQL uses hash joins for the activity tables (planned_activity and employee_activity). This is chosen because these tables are being joined on non-unique columns where creating a hash table in memory (Buckets: 1024) is faster than looping through an index.

The statistics per relation

The accessed tables are: planned_activity and employee_activity (scanned sequentially twice: once for the main query and once for the helper), teaching_activity, course_instance and course_layout, employee, person and job_title.

We see multiple sequential scans here (5 in total). This is acceptable in this specific context because we are filtering for a single course instance (ID=1).

Performance insight step by step

Please see our execution plan tree in our github.

The query execution is highly efficient with an execution time of 1.234 ms. We observed that the planning time (3.211 ms) was actually higher than the execution time, bringing the total latency to roughly 4.45 ms. This is expected for a small test database: the optimizer spends more time figuring out how to join 8 tables than it takes to actually read the few existing rows.

Adding an index

The heaviest operations in this plan are the sequential scans on the `planned_activity` table.

Currently, the query filters by `instance_id`. This filter removes 89 rows and keeps 11. While this is fast now, if the `planned_activity` table grows to millions of rows, scanning the entire table to find the few rows will become a major problem. We also noticed that the execution plan scans this table twice: once for the main query calculation and again inside the CTE (`teacher_counts`) to count the teachers. Since both the main query and the CTE need to find the same specific course instance, the lack of an index effectively doubles the unnecessary workload.

Therefore, the most effective optimization for future growth is to index the foreign key:

```
CREATE INDEX idx_planned_activity_instance
ON planned_activity (instance_id);
```

This single index would optimize both scans at the same time, allowing both the main query and the helper CTE to jump directly to the relevant rows without reading the entire table.

Exploring views

Since this query contains complex formulas and involves joining 8 different tables we should use a view.

```
CREATE VIEW v_course_allocation_breakdown AS
SELECT ... ..
```

Creating this view would not improve execution speed, but it would ensure consistency.

5.3 Analysis of the total allocated hours query

The statistics per node type

The operations executed for this query are:

- aggregate (GroupAggregate) = 1 (performs grouping and sum calculations per teacher and course instance);
- sort = 1 (sorts the results by course code, instance, and hp as required by `GROUP BY` and `ORDER BY`);
- nested loop = 6 (correspond to the joins along the chain `planned_activity` → `course_instance` → `employee_activity` → `employee` → `person`, then to `teaching_activity` and finally `course_layout`);
- seq scan = 1 (on `planned_activity` as the starting relation and row source);
- index scan = 4 (on `course_instance`, `employee`, `person`, `teaching_activity`, and `course_layout` for efficient key lookups);
- index-only scan = 1 (on `employee_activity` using its primary key, with a few heap fetches);
- memoize = 1 (to cache repeated `course_instance` lookups by `pa.instance_id`).

Same as for the first query, PostgreSQL uses index scans for all key-based joins, so it can jump directly to matching rows instead of scanning whole tables. The only full table read is the sequential scan on `planned_activity`, which is acceptable given the small size. Quicksort is used to order the (very few) intermediate rows before grouping, which is over-engineered at this scale but will become useful as the database grows. The Memoize node again avoids redundant index scans on `course_instance` when the same `instance_id` appears multiple times.

The statistics per relation

The accessed tables are: `planned_activity` (scanned sequentially once), `course_instance` (index scan via PK, with filter on current year), `employee_activity` (index-only scan by PK), `employee` (index scan by `employment_id`), `person` (index scan by `person_id`, plus filter on teacher name), `teaching_activity` (index scan by `teaching_activity_id`), and `course_layout` (index scan by FK `course_layout_id`).

This matches expectations: each planned activity is joined to an instance, then to the employee(s) performing it, and finally to teacher and activity metadata. The sequential scan on `planned_activity` is still fine for a small fact table; if it grows, an index by `instance_id` might help. The memoization hit/miss (80 hits, 20 misses) again shows that many planned activities share the same `instance_id`, so caching those lookups saves repeated work.

Performance insight step by step

The planner is very similar to the first query. The planning took 10ms and executed in 4ms.

Adding an index

The heavier parts for this query are:

- the seq scan on `planned_activity`;
- repeated lookups into `course_instance` by `instance_id` (already indexed and memoized);
- index(-only) lookups on `employee_activity`, `employee`, `person`, `teaching_activity`, `course_layout` (all via PKs or FKs).

As before, a multi-column index on `course_instance(instance_id, study_year)` would not help much here, because `instance_id` is already unique and the `study_year` filter is applied after that lookup. If `planned_activity` becomes large, the most promising index is again:

```
CREATE INDEX idx_planned_activity_instance ON planned_activity (instance_id);
```

This would make it possible (for other queries) to start from `course_instance` and find all planned activities by index rather than by sequential scan, and could also allow the planner to choose a different join order if that becomes beneficial.

Exploring views

A view for this teacher-specific allocated-hours query would mainly help with reuse and clarity, for example:

```
CREATE VIEW teacher_allocated_hours_current_year AS
-- same SELECT ... FROM ... WHERE ... GROUP BY ...;
```

Then reports could simply query:

```
SELECT *
FROM teacher_allocated_hours_current_year
WHERE "Teacher's Name" = 'Filip Eriksson';
```

This does not directly speed up execution (PostgreSQL inlines simple views), but it centralizes the formulas and joins in one place and makes it **easier to maintain** or adapt the logic for multiple teachers or years.

5.4 Analysis of the teacher course load query

The statistics per node type

The operations executed for this query are:

- aggregate (GroupAggregate) = 1 (groups the results by teacher and performs the `COUNT(DISTINCT)` on course instances);
- sort = 1 (sorts the joined rows by employment ID and name to prepare for the grouping);

- nested loop = 4 (correspond to the joins along the chain `planned_activity → course_instance → employee_activity → employee → person`);
- seq scan = 1 (on `planned_activity` as the starting relation);
- index scan = 3 (one each for `course_instance`, `employee` and `person` for efficient key lookups);
- index-only scan = 1 (on `employee_activity` using its primary key, retrieving data without visiting the table heap);
- memoize = 1 (to cache `course_instance` lookups).

It again uses nested loops on the `planned_activity` table. A good optimization here is the index only scan on `employee_activity`. Since the query only needed the IDs to link the planned activity to the employee, PostgreSQL was able to get all necessary information directly from the index structure without reading the actual table rows.

The statistics per relation

The accessed tables are: `planned_activity` (scanned sequentially once), `course_instance` (index scan filtered by year and the complex study period logic), `employee_activity`, `employee` (index scan by `employment_id`), and `person` (index scan by `person_id`).

The filtering logic for the "Current Period" (the complex CASE statement regarding dates) is applied efficiently as a filter during the index scan on `course_instance`. This means rows that do not match the current period are avoided early in the process.

Performance insight step by step

The execution flow begins by reading `planned_activity`. For every row found, it looks up the corresponding `course_instance`. If that instance matches the current year and period (P2), it proceeds to find the employee and person details. The final steps involve sorting the gathered data by teacher and then performing the `GroupAggregate` to count the `DISTINCT` instance IDs.

Adding an index

The heavy work currently involves:

- The sequential scan on the `planned_activity` table;
- The complex date calculation and filtering on the `course_instance` table.

Since the query relies heavily on filtering `course_instance` by date, one might initially consider an index on the `study_period` column. However, this would not work effectively because our query does not use a stored value for the period. Instead, it calculates the period dynamically at runtime using a CASE statement with `CURRENT_DATE`.

Therefore, if the dataset grows, the most promising optimization is to create an index on the foreign key in the `planned_activity` table:

```
CREATE INDEX idx_planned_activity_instance
ON planned_activity (instance_id);
```

Exploring views

We analyzed whether a view would be beneficial for this query. A standard view would improve code readability by hiding the complex date logic, but it provides no performance benefit since the underlying query is executed every time.

We also considered a materialized view to cache the results. Its efficiency would depend on the frequency at which this query would be run.

6 Higher grade discussion

In this section, we discuss alternate designs for our OLAP based on the frequency of our queries as well as new queries. To view the view we built for these improved queries, go to our build script in sql: the last lines define the view.

To fully understand and compare the analyses provided by PostgreSQL, we learned that:

- buffers are the number of 8kB data pages the query touched, and from where
- shared buffers are pages came that from PostgreSQL's shared buffer cache (its in-memory cache)
- "Shared hit=25 means": 25 pages were found already in that cache (they'r a cache hit), so PostgreSQL did not have to read them from disk⁵

OBS: given our small data set, it was very difficult to notice any observable improvements made by our design choices : *if they were improvements were they do to PostgreSQL built-in memory ? caches ? and if they were no improvements was it because the overhead of our new indices or views took over ? would more benefits be seen with a larger dataset ?* We did not have sufficient data to answer these questions.

6.1 Indices for our workload queries

About indexes, they help the most when a query runs often and filters or joins on columns that are not already primary-key/foreign-key indexed. They also help a lot when they involve many rows, which is not really our case since our database is so small (but will likely grow in real life).

Tradeoffs between index maintenance cost and query performance:

When to Use Many Indexes:

1. When we mostly read data and add or change data rarely
2. When our tables are big and our queries skip most data (over 90%)
3. When columns are used for linking tables (foreign keys) or joined frequently

When to Keep Indexes Few or Small:

1. When we write or update data a lot and speed of writes is important
2. When tables often get big inserts or deletes all at once
3. When columns have few unique values that don't help quickly find rows
4. When queries vary a lot and you can't predict which columns to index

This means indexes make searching faster but slowing down data changes. More indexes help reads but add extra work on writes and take extra disk space.

The queries in bold in table 7 show the strong candidates for indexing based on usage frequency and our model. That means those that have priority to having an index.

⁵Consequently, more shared hits means more work satisfied from RAM, less disk I/O so usually a better performance

query	main tables	heavy scans	filters	potentially useful indexes
planned hours per course instance	ci, pa, ta, cl	seq scan on pa		planned_activity(instance_id)
actual hours per teacher per course	e, p, ea, pa, ci, cl, ta		for teacher, current year	person(first_name, last_name) planned_activity(instance_id)
teacher load per period	e, p ci, pa, ta			course_instance(study_year, study_period)
Course instances with total planned hours vs total allocated hours				
Teachers allocated to more than N courses in current period	e, p, ea, pa, ci	seq scan on ea, pa, ci	on ci for current period	course_instance(study_period, study_year, instance_id)

Table 7: Recap of workload queries and their useful indices, bold query means strong candidate

- ci = course instance
- pa = planned activity
- ta = teaching activity
- cl = course layout
- e = employee
- p = person

6.2 Materialized views

Tradeoffs between types of views: Overall, we want normal views for real-time, low-latency tolerance but not for read-heavy, aggregative or latency-sensitive workloads.

	Normal views	Materialized views
pros	always up to date; logical abstraction without storage overhead; simplifies complex queries	precomputed and stored for fast read access; significant performance gain for expensive queries
cons	performance cost at query time (they recompute underlying queries); no persisted data	storage overhead; stale data without refresh; refresh cost (time/resources) can be high
best for queries	1,3 and 4	2 and 5

Table 8: Summary of tradeoffs between normal and materialized views

6.3 Improving our queries and analyzing improvements

Here is a summary of our decisions and our reasons:

Query	Note on the query	Suggested strategy
Planned hours per course instance	Frequently executed but needs real-time data.	A view to get abstraction and data freshness and an index to speed up join.
Actual allocated hours per teacher per course instance	Highly frequent query, could benefit from pre-aggregated results.	A materialized view.
Teacher load per period	Moderate frequency, needs real-time data, benefits from efficient filtering.	An index (most important) and a view (to avoid CASE, for easier maintenance)
Course instances with variance 15%	Low frequency, complex calculation needs real-time data	A normal view would suffice
Teachers allocated to N courses	Highest frequency, simple aggregation pattern perfect for materialization	A materialized view.

6.3.1 For the first query

Using the view offloads the sums and business logic (including formulas for "Examination" and "Administration") into **one reusable layer**. Every query of planned hours now just needs to aggregate over this view, making SQL simpler. Adding the index `planned_activity(instance_id)` directly speeds up queries that join on `instance_id` (regardless of if it's on views or not, since the view's main aggregation groups by `instance_id`). This is why we thought it best to **use both a new index and a normal view**.

	Original query	Using the view and index
Top node	GroupAggregate	GroupAggregate
Sort	yes (quicksort)	yes (quicksort)
Main join type	Nested Loop chain	Hash Join + Nested Loops
Start table	<code>planned_activity</code> (Seq Scan)	<code>planned_activity</code> (Seq Scan in view)
Extra work	memoize on <code>course.instance</code>	Append + subqueries from view
Estimated cost	23.06..23.19	326.96..327.88
Actual time	1.834 ms	1.945 ms
Shared buffers hit	165	25

Table 9: Comparison of original query vs. view-based query for the first higher grade query

Though they have very similar runtimes, the theoretically "improved" queries performed worse (probably due to a more complex plan due to the UNION ALL view, but still efficient for this data size).

6.3.2 For the second query

This query is supposed to calculate the actual allocated hours per teacher for a specific course instance. As previously analyzed, the original query is expensive because it requires a CTE to calculate the `teacher_counts`, followed by a main query joining eight different tables. Considering the complexity of the joins and the requirement to run this query 12 times per day, we decided to implement a materialized view. We created the materialized view `mv_teacher_allocated_hours` to pre-calculate these allocations for all course instances. This allows the database to compute the heavy joins once and store the result physically. To get maximum speed, we paired the materialized view with a composite index on `(instance_id, study_year)`.

	Original query	Using the view and index
Top node	GroupAggregate	Seq Scan
Sort	yes (quicksort)	no
Main join type	Nested Loop + Hash Join	none
Start table	employee_activity (Seq Scan)	mv_teacher_alloc... (Seq Scan)
Extra work	-	Filter
Estimated cost	50.95..52.58	0.00..3.82
Planning time	3.211 ms	2.416 ms
Execution time	1.234 ms	0.121 ms
Shared buffers hit	75	2

Table 10: Comparison of original query vs. view-based query for the second higher grade query

6.3.3 For the third query

This query outputs the total number of hours a specific teacher has for a specific period (we chose Filip Eriksson for P2 for example). We decided to **reuse the v_full_course_workload view**: it already has all hours we are interested in (planned activities + admin + exam) as total.teachers.hours per instance and activity and this avoids repeating formulas/CASE in every query. We **also use a course_instance(study_year, study_period) index**: we filter on both study_year (current) and a specific STUDY_PERIOD argument, so the composite index lets PostgreSQL jump directly to the relevant course_instance rows.

Employment ID	Teacher's Name	Period	Year	Total Hours
12	Filip Eriksson	P2	2025	837.9999890327454

Table 11: Result output of our query on teacher load per period (first higher grade query)

This plan showed our teacher-load query using both the view and the new index effectively. The Append plus subqueries correspond to v_full_course_workload, centralizing workload logic while still expanding to cheap scans at this data size. The key win from indexing is the Index Scan using idx_planned_activity_instance on planned_activity, which turns “get activities by instance_id” into fast index lookups instead of repeated full scans. Although total runtime is still only about 3 ms, this indexed nested-loop pattern will scale better as planned_activity grows, while the view keeps the SQL for reporting queries much simpler.

6.3.4 For the fourth query

For this query, which checks for course instances with planned vs actual variance > 0.15 variance and runs only 3 times a day, we decided to implement a standard view

(v_course_variance). To make it efficient, we decided to reuse our previous work (in query 2) so instead of recalculating the "actual hours" from scratch with a 8 table join, the view simply takes that data from the mv_teacher_allocated_hours materialized view we created . We then join this pre-calculated data with a real-time calculation of "planned hours" (from query 1) to compute the percentage difference. We completely avoid the expensive step of joining 8 different tables to find the allocated hours which is very nice optimization. The only trade-off is that data depends on the materialized view being fresh.

	Query 4
Top node	Hash Join
Sort	yes (quicksort in subquery)
Main join type	Hash Join
Start table	mv_teacher_allocated_hours (Seq Scan)
Extra work	Join Filter (variance > 0.15)
Estimated cost	26.40..26.90
Execution time	0.528 ms
Shared buffers hit	167

Table 12: Performance profile of the variance reporting query using the views

6.3.5 For the fifth query

A basic implementation of this query (whose goal is to find the teachers allocated to more than N courses in the current period) could be :

```
SELECT  e.employment_id,
        p.first_name || ' ' || p.last_name AS teacher_name,
        COUNT(DISTINCT ci.instance_id) AS course_count

FROM    employee e
        JOIN person p ON e.person_id = p.person_id
        JOIN employee_activity ea ON e.employment_id = ea.employment_id
        JOIN planned_activity pa ON ea.planned_activity_id = pa.planned_activity_id
        JOIN course_instance ci ON pa.instance_id = ci.instance_id
-- WHERE clause for current year and period using EXTRACT()
GROUP BY e.employment_id, p.first_name, p.last_name
HAVING COUNT(DISTINCT ci.instance_id) > 1 -- N parameter
ORDER BY course_count DESC;
```

However, this query will be run 20 times a day, it could be optimized because right now, it uses full table scans: PostgreSQL scans every row in e, p, ea, pa and ci. Firstly, we could optimize **using an index** such as : `CREATE INDEX idx_ci_period_year ON`

`course_instance(study_period, study_year, instance_id)`. You can view this suggestion on this link.

But since this query will be run the most frequently, a materialized view also seems like a smart choice. You can read this second implementation, with its different strategy, on this link. We decided to have data being stale for about 3 hours every day, assuming it was acceptable for the customer. This requires a **refresh rate of 4 times per day**.

	Without Index	With Index	With MatView
Top node	GroupAggregate	GroupAggregate	Sort
Sort	yes (quicksort)	yes (quicksort)	yes (quicksort)
Main join type	Nested loops+hash	Nested loops+hash	none (single table)
Start table	ci (Seq Scan)	ci (Seq Scan)	mv (Seq Scan)
Estimated cost	11.76..11.77	11.76..11.77	5.16..5.16
Actual time	0.061..0.079 ms	0.045..0.056 ms	0.021..0.041 ms
Shared buffers hit	3	3 (plus 1 read)	1
Planning time	0.931 ms	1.380 ms	0.118 ms
Execution time	0.163 ms	0.123 ms	0.041 ms

Table 13: Comparison of execution plans plain, with index and with matview for the fifth higher grade query shows significant improvement with materialized view

- **Performance Improvement:** Execution time reduced from **0.163ms** → **0.041ms** (75% faster) compared to the original query
- **Resource Efficiency:** Only **1 buffer hit** vs 3 previously shows minimal I/O overhead
- **Faster Planning:** Planning time reduced from **1.380ms** → **0.118ms** due to simpler query structure
- **Filter Efficiency:** The sequential scan filters **79 rows down to 0**, demonstrating effective data reduction
- **Cost Reduction:** Estimated cost decreased from **11.76** → **5.16** units, indicating significantly less computational work

Trade-off: The improved performance comes at the cost of data freshness: we will have to periodically refresh the matview.

6.4 Self assessment

Criteria	Notes
Are views and materialized views used in all queries that benefit from using them?	We have not used views and materialized views in any query however they were suggested in discussions since our analysis indicates that views would be highly beneficial for query 2 and query 4.
Can any query be made easier to understand by storing part of it in a view?	Every query could potentially benefit from using views, depending on how the database will be interacted with. Storing complex logic in views will make data access faster for the user, though it can introduce some management overhead. It all depends on customer needs.
Can performance be improved by using a materialized view?	We could not measure any improvements, but it could be because our database is so small and execution times are under 2ms. We did see an improvement on performance for the higher grade query 5.
Did you change the database design to simplify these queries?	We did not change the database to simplify the queries.
Is there any correlated subquery, that is, a subquery using values from the outer query?	No, there are no correlated subqueries in either query. A correlated subquery would be a SELECT inside SELECT/WHERE that references columns from the outer query. We purposefully avoided this.
Are there unnecessarily long and complicated queries? Are you for example using a UNION clause where it's not required?	We haven't used UNION in our queries. We tried to write the queries so they are easy to follow however in it possible that they can be written simpler more efficiently. However, we did use UNION in our view for a teacher's workload.

Table 14: Self-assessment based on the task's criteria

7 Comments About the Course

The course gave use sufficient information to write query trees, relational algebra expressions and finally SQL queries. We had finished the previous assignment with a view that gave columns on administration and examination hours since we thought this was required for task 1. But this led us astray for task 2. We started with the view, then later did without it. Overall the explanations on Admin and Exam hours were not sufficiently clear on this point.